

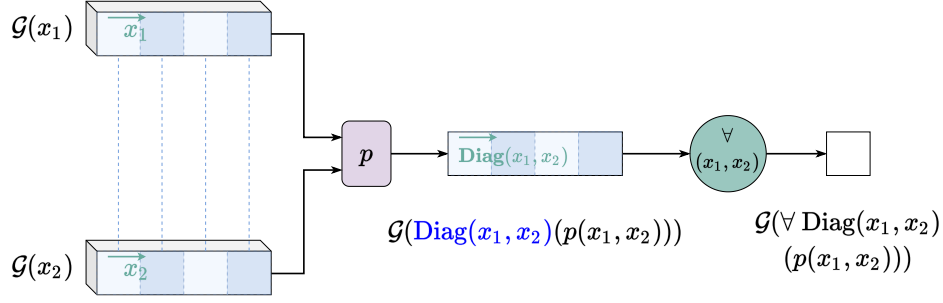


Figure 4: Diagonal Quantification: $\text{Diag}(x_1, x_2)$ quantifies over specific tuples only, such that the i -th tuple contains the i -th instances of the variables x_1 and x_2 in the groundings $\mathcal{G}(x_1)$ and $\mathcal{G}(x_2)$, respectively. $\text{Diag}(x_1, x_2)$ assumes, therefore, that x_1 and x_2 have the same number of instances as in the case of samples x_1 and their labels x_2 in a typical supervised learning tasks.

$$\text{Agg}(\forall)_{j=1, \dots, |\mathcal{G}(y)|} \quad \text{Agg}(\exists)_{i=1, \dots, |\mathcal{G}(x)| \text{ s.t. } \mathcal{G}(\text{age}(x))_i > \mathcal{G}(\text{age}(y))_j} \quad \mathcal{G}(\text{parent}(x, y))_{i,j}$$

The evaluation of which tuple is safe is purely symbolic and non-differentiable. Guarded quantifiers operate over only a subset of the variables, when this symbolic knowledge is crisp and available. More generally, in what follows, m is a symbol representing the condition, which we shall call a *mask*, and $\mathcal{G}(m)$ associates a function⁶ returning a Boolean to m .

$$\mathcal{G}(Q \ x_1, \dots, x_h : m(x_1, \dots, x_n)(\phi))_{i_{h+1}, \dots, i_n} \stackrel{\text{def}}{=} \text{Agg}(Q)_{i_1=1, \dots, |\mathcal{G}(x_1)|} \quad \mathcal{G}(\phi)_{i_1, \dots, i_h, i_{h+1}, \dots, i_n} \quad (6)$$

$$\vdots$$

$$i_h=1, \dots, |\mathcal{G}(x_h)| \text{ s.t. } \mathcal{G}(m)(\mathcal{G}(x_1)_{i_1}, \dots, \mathcal{G}(x_n)_{i_n})$$

Notice that the semantics of a guarded sentence $\forall x : m(x)(\phi(x))$ is different than the semantics of $\forall x(m(x) \rightarrow \phi(x))$. In crisp and traditional FOL, the two statements would be equivalent. In Real Logic, they can give different results. Let $\mathcal{G}(x)$ be a sequence of 3 values, $\mathcal{G}(m(x)) = (0, 1, 1)$ and $\mathcal{G}(\phi(x)) = (0.2, 0.7, 0.8)$. Only the second and third instances of x are safe, that is, are in the masked subset. Let $\rightarrow$ be defined using the Reichenbach operator $I_R(a, b) = 1 - a + ab$ and $\forall$ be defined using the mean operator. We have $\mathcal{G}(\forall x(m(x) \rightarrow \phi(x))) = \frac{1+0.7+0.8}{3} = 0.833\dots$ whereas $\mathcal{G}(\forall x : m(x)(\phi(x))) = \frac{0.7+0.8}{2} = 0.75$. Also, in the computational graph of the guarded sentence, there are no gradients attached to the instances that do not verify the mask. Similarly, the semantics of $\exists x : m(x)(\phi(x))$ is not equivalent to that of $\exists x(m(x) \wedge \phi(x))$.

⁶In some edge cases, a masking may produce an empty sequence, e.g. if for some value of $\mathcal{G}(y)$, there is no value in $\mathcal{G}(x)$ that satisfies $\text{age}(x) > \text{age}(y)$, we resort to the concept of an *empty semantics*: $\forall$ returns 1 and $\exists$ returns 0.

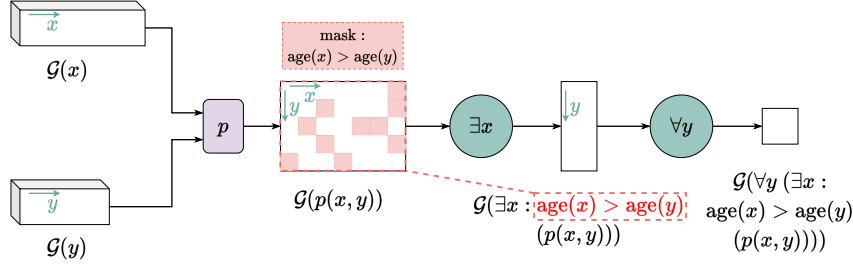


Figure 5: Example of Guarded Quantification: One can filter out elements of the various domains that do not satisfy some condition before the aggregation operators for $\forall$ and $\exists$ are applied.

2.4. Stable Product Real Logic

It has been shown in [69] that not all first-order fuzzy logic semantics are equally suited for gradient-descent optimization. Many fuzzy logic operators can lead to vanishing or exploding gradients. Some operators are also *single-passing*, in that they propagate gradients to only one input at a time.

In general, the best performing symmetric configuration⁷ for the connectives uses the product t-norm T_P for conjunction, its dual t-conorm S_P for disjunction, standard negation N_S , and the Reichenbach implication I_R (the corresponding S-Implication to the above operators). This subset of Real Logic where the grounding of the connectives is restricted to the product configuration is called *Product Real Logic* in [69]. Given a and b two truth-values in $[0, 1]$:

$$\neg : N_S(a) = 1 - a \quad (7)$$

$$\wedge : T_P(a, b) = ab \quad (8)$$

$$\vee : S_P(a, b) = a + b - ab \quad (9)$$

$$\rightarrow : I_R(a, b) = 1 - a + ab \quad (10)$$

Appropriate aggregators for $\exists$ and $\forall$ are the generalized mean A_{pM} with $p \geq 1$ to approximate the existential quantification, and the generalized mean w.r.t. the error A_{pME} with $p \geq 1$ to approximate the universal quantification. They can be understood as a smooth maximum and a smooth minimum, respectively. Given n truth-values $a_1, \dots, a_n$ all in $[0, 1]$:

$$\exists : A_{pM}(a_1, \dots, a_n) = \left(\frac{1}{n} \sum_{i=1}^n a_i^p \right)^{\frac{1}{p}} \quad p \geq 1 \quad (11)$$

$$\forall : A_{pME}(a_1, \dots, a_n) = 1 - \left(\frac{1}{n} \sum_{i=1}^n (1 - a_i)^p \right)^{\frac{1}{p}} \quad p \geq 1 \quad (12)$$

A_{pME} measures the power of the deviation of each value from the ground truth 1. With $p = 2$, it is equivalent to $1 - \text{RMSE}(\mathbf{a}, \mathbf{1})$, where RMSE is the root-mean-square error, $\mathbf{a}$ is the vector of truth-values and $\mathbf{1}$ is a vector of 1's.

⁷We define a symmetric configuration as a set of fuzzy operators such that conjunction and disjunction are defined by a t-norm and its dual t-conorm, respectively, and the implication operator is derived from such conjunction or disjunction operators and standard negation (c.f. Appendix B for details). In [69], van Krieken et al. also analyze non-symmetric configurations and even operators that do not strictly verify fuzzy logic semantics.

The intuition behind the choice of p is that the higher that p is, the more weight that A_{pM} (resp. A_{pME}) will give to *true* (resp. *false*) truth-values, converging to the $\max$ (resp. $\min$) operator. Therefore, the value of p can be seen as a hyper-parameter as it offers flexibility to account for outliers in the data depending on the application.

Nevertheless, *Product Real Logic* still has the following gradient problems: $T_P(a, b)$ has vanishing gradients on the edge case $a = b = 0$; $S_P(a, b)$ has vanishing gradients on the edge case $a = b = 1$; $I_R(a, b)$ has vanishing gradients on the edge case $a = 0, b = 1$; $A_{pM}(a_1, \dots, a_n)$ has exploding gradients when $\sum_i (a_i)^p$ tends to 0; $A_{pME}(a_1, \dots, a_n)$ has exploding gradients when $\sum_i (1 - a_i)^p$ tends to 0 (see Appendix C for details).

To address these problems, we define the projections π_0 and π_1 below with ϵ an arbitrarily small positive real number:

$$\pi_0 : [0, 1] \rightarrow]0, 1] : a \rightarrow (1 - \epsilon)a + \epsilon \quad (13)$$

$$\pi_1 : [0, 1] \rightarrow [0, 1[: a \rightarrow (1 - \epsilon)a \quad (14)$$

We then derive the following stable operators to produce what we call the *Stable Product Real Logic* configuration:

$$N'_S(a) = N_S(a) \quad (15)$$

$$T'_P(a, b) = T_P(\pi_0(a), \pi_0(b)) \quad (16)$$

$$S'_P(a, b) = S_P(\pi_1(a), \pi_1(b)) \quad (17)$$

$$I'_R(a, b) = I_R(\pi_0(a), \pi_1(b)) \quad (18)$$

$$A'_{pM}(a_1, \dots, a_n) = A_{pM}(\pi_0(a_1), \dots, \pi_0(a_n)) \quad p \geq 1 \quad (19)$$

$$A'_{pME}(a_1, \dots, a_n) = A_{pME}(\pi_1(a_1), \dots, \pi_1(a_n)) \quad p \geq 1 \quad (20)$$

It is important noting that the conjunction operator in stable product semantics is not a T-norm⁸. $T'_P(a, b)$ does not satisfy identity in $[0, 1]$ since for any $0 \leq a < 1$, $T'_P(a, 1) = (1 - \epsilon)a + \epsilon \neq a$, although ϵ can be chosen arbitrarily small. In the experimental evaluations reported in Section 4, we find that the adoption of the stable product semantics is an important practical step to improve the numerical stability of the learning system.

3. Learning, Reasoning, and Querying in Real Logic

In Real Logic, one can define the tasks of *learning*, *reasoning* and *query-answering*. Given a Real Logic theory that represents the knowledge of an agent at a given time, *learning* is the task of making generalizations from specific observations obtained from data. This is often called inductive inference. *Reasoning* is the task of deriving what knowledge follows from the facts which are currently known. *Query answering* is the task of evaluating the truth value of a certain logical expression (called a query), or finding the set of objects in the data that evaluate a certain expression to *true*. In what follows, we define and exemplify each of these tasks. To do so, we first need to specify which types of knowledge can be represented in Real Logic.

⁸Recall that a T-norm is a function $T : [0, 1] \times [0, 1] \rightarrow [0, 1]$ satisfying commutativity, monotonicity, associativity and identity, that is, $T(a, 1) = a$.